

E-Commerce Backend API Documentation

Base URL: `http://localhost:3007/v1`

Currency (configurable): EGP (Egyptian Pound)

Auth Format: `Authorization: <role> <token>` (e.g. `Authorization: user
eyJhbGciOiJIUzI1NiIs...`)

Table of Contents

1. Authentication
2. User Profile
3. Phone Verification (WhatsApp OTP)
4. Categories
5. Subcategories
6. Products
7. Cart
8. Wishlist
9. Orders & Payments
10. Security Logs
11. Error Handling & Auth System
12. Database Models

1. Authentication (`/v1/auth`)

1.1 POST `/v1/auth/signup` — Register a new user

Auth: None

Rate Limit: None

Request Body (JSON):

```
{
  "name": "john_doe",
  "email": "john@example.com",
  "password": "123456",
  "confirmPassword": "123456",
  "role": "user",
  "phone": "+201234567890"
}
```

Field	Type	Required	Description
name	string	Yes	Unique username
email	string	Yes	Unique email address
password	string	Yes	Min 6, max 20 characters
confirmPassword	string	Yes	Must match <code>password</code>
role	string	No	One of: <code>user</code> , <code>admin</code> , <code>superAdmin</code> (default: <code>user</code>)
phone	string	No	International format (e.g. <code>+2126xxxxxxxx</code>)

Success Response `201 Created`:

```
{
  "message": "Signup successful – verification email sent"
}
```

The user is created with `isActive: false`. A verification email is sent. The user **cannot login** until email is verified.

Error Responses:

- `400` — `{ "message": "Email already exists" }`
- `400` — `{ "message": "Name already exists" }`
- `400` — `{ "message": "Passwords do not match" }`
- `400` — `{ "message": "validation error", "error": [...] }` (Joi validation details)

1.2 POST `/v1/auth/login` — Login with email/password

Auth: None

Rate Limit: None

Request Body (JSON):

```
{
  "email": "john@example.com",
  "password": "123456"
}
```

Success Response `200 OK`:

```
{
  "message": "Login successful",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
    "refreshToken": "eyJhbGciOiJIUzI1NiIs..."
  }
}
```

- `accessToken` expires in **30 minutes** (configurable via `ACCESS_TOKEN`)
- `refreshToken` expires in **1 year** (configurable via `REFRESH_TOKEN`)
- Tokens are signed with **role-specific secrets** (`SIGNATURE_USER` , `SIGNATURE_ADMIN` , `SIGNATURE_SUPER_ADMIN`)

How to use tokens in subsequent requests:

```
Authorization: user eyJhbGciOiJIUzI1NiIs...
```

Replace `user` with the actual role (`admin` , `user` , `superAdmin`).

Error Responses:

- `400` — { "message": "Invalid email or password" }
- `400` — { "message": "This account uses Google login" } (if user was created via Google OAuth and has no password)

1.3 GET `/v1/auth/google/signup` — Google OAuth Signup

Auth: None

Rate Limit: None

Query Parameters:

Param	Type	Default	Description
<code>role</code>	string	<code>user</code>	Role to assign to new user: <code>user</code> , <code>admin</code> , <code>superAdmin</code>

Behavior: Redirects the browser to Google's OAuth consent screen. After user grants permission, Google redirects to the callback URL.

Callback: `GET /v1/auth/google/signup/callback`

Success Response `200 OK` (from callback):

```
{
  "message": "Google signup successful",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
    "refreshToken": "eyJhbGciOiJIUzI1NiIs... ",
    "user": { "id": 1, "role": "user" }
  }
}
```

Error Responses (callback):

- `400` — `{ "message": "An account with this email already exists. Please login instead." }`
 - `400` — `{ "message": "No email found from Google account" }`
 - `400` — `{ "message": "Role must be one of: admin, user, superAdmin" }` (if invalid role in query)
-

1.4 GET `/v1/auth/google/login` — Google OAuth Login

Auth: None

Behavior: Redirects to Google's OAuth consent screen for existing users.

Callback: `GET /v1/auth/google/login/callback`

Success Response `200 OK` (from callback):

```
{
  "message": "Google login successful",
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
    "refreshToken": "eyJhbGciOiJIUzI1NiIs... ",
    "user": { "id": 1, "role": "user" }
  }
}
```

Error Responses (callback):

- `400` — `{ "message": "No account found with this email. Please signup first." }`
 - `400` — `{ "message": "Please verify your email first." }` (if user registered via email but hasn't verified yet)
-

1.5 GET /v1/auth/verify-email/:token — Verify Email

Auth: None

Rate Limit: None

Path Parameters:

Param	Type	Description
token	string	JWT token from verification email link

Success Response 200 OK :

```
{
  "message": "Email verified successfully"
}
```

Or if already verified:

```
{
  "message": "Email already verified"
}
```

Error Responses:

- 400 — { "message": "Invalid or expired token" }

1.6 POST /v1/auth/resend-verify-email — Resend Verification Email

Auth: None

Rate Limit: 1 request per 60 seconds

Request Body (JSON):

```
{
  "email": "john@example.com"
}
```

Success Response 200 OK :

```
{
  "message": "Verification email resent"
}
```

Error Responses:

- `400` — `{ "message": "User not found" }`
 - `200` — `{ "message": "Email already verified" }` (if already active)
-

1.7 POST `/v1/auth/forgot-password` — Forgot Password

Auth: None

Rate Limit: 1 request per 60 seconds

Request Body (JSON):

```
{
  "email": "john@example.com"
}
```

Success Response `200 OK` (always returns this message regardless of whether email exists — security best practice):

```
{
  "message": "If the email exists, a reset link has been sent"
}
```

How it works:

1. Generates a JWT token (expires 15 minutes)
 2. Stores token in Redis with key `reset:<token>` (expires 900 seconds)
 3. Sends email with reset link: `{BASE_URL}/v1/auth/reset-password/{token}`
-

1.8 POST `/v1/auth/reset-password/:token` — Reset Password

Auth: None

Rate Limit: 1 request per 60 seconds

Path Parameters:

Param	Type	Description
<code>token</code>	string	JWT token from reset password email link

Request Body (JSON):

```
{
  "password": "newpassword123",
  "confirmPassword": "newpassword123"
}
```

Success Response `200 OK` :

```
{
  "message": "Password reset successful"
}
```

Error Responses:

- `400` — { "message": "Passwords do not match" }
- `400` — { "message": "Invalid or expired token" }
- `400` — { "message": "User not found" }

2. User Profile (`/v1/user`)

2.1 GET `/v1/user/profile` — Get My Profile

Auth: Required (any authenticated user)

Rate Limit: None

Headers:

```
Authorization: user eyJhbGciOiJIUzI1NiIs...
```

Success Response `200 OK` :

```
{
  "message": "Profile",
  "data": {
    "id": 1,
    "name": "john_doe",
    "email": "john@example.com",
    "isActive": true,
    "imageUrl": "https://res.cloudinary.com/.../profiles/abc123.jpg",
    "role": "user",
    "phone": "+201234567890",
    "googleId": null,
    "isBlocked": false,
    "createdAt": "2026-06-14T10:00:00.000Z",
    "updatedAt": "2026-06-14T10:00:00.000Z"
  }
}
```

Note: `password` field is excluded from response.

Error Responses:

- `401` — `{ "message": "Login required" }`
- `404` — `{ "message": "User not found" }`

2.2 PUT `/v1/user/profile` — Update Profile

Auth: Required (any authenticated user)

Rate Limit: None

Content-Type: `multipart/form-data`

Request Fields:

Field	Type	Required	Description
<code>phone</code>	string	No	International format (e.g. <code>+2126xxxxxxxx</code>)
<code>image</code>	file	No	Profile picture (JPEG, PNG, GIF, WebP — max 2MB)

Success Response `200 OK` :

```
{
  "message": "Profile updated",
  "data": {
    "id": 1,
    "name": "john_doe",
    "email": "john@example.com",
    "imageURL": "https://res.cloudinary.com/.../profiles/new123.jpg",
    "phone": "+201234567890",
    "role": "user",
    ...
  }
}
```

How it works:

- If a new image is uploaded, the old image is **deleted from Cloudinary** before uploading the new one.
- Image validation is done via `file-type` package (checks actual file bytes, not just extension).

Error Responses:

- `400` — `{ "message": "No fields to update" }`
- `400` — `{ "message": "Invalid file type: ... Only JPEG, PNG, GIF, and WebP are allowed." }`

2.3 POST `/v1/user/upload-avatar` — Upload Avatar

Auth: Required (any authenticated user)

Rate Limit: None

Content-Type: `multipart/form-data`

Request Fields:

Field	Type	Required	Description
<code>image</code>	file	Yes	Avatar image (JPEG, PNG, GIF, WebP — max 2MB)

Success Response `200 OK` :

```
{
  "message": "Avatar uploaded",
  "data": {
    "imageURL": "https://res.cloudinary.com/.../profiles/abc123.jpg"
  }
}
```

Error Responses:

- `400` — `{ "message": "No image provided" }`

3. Phone Verification (`/v1/phone`)

3.1 POST `/v1/phone/send-otp` — Send OTP via WhatsApp

Auth: Required (user role)

Rate Limit: None

Request Body (JSON):

```
{
  "phone": "+212612345678"
}
```

Field	Type	Required	Description
<code>phone</code>	string	Yes	International format (e.g. <code>+2126xxxxxxxx</code>)

How it works:

- Generates a 6-digit numeric OTP (e.g. `482931`)

2. Stores OTP in **Redis** with key `otp:+212612345678` , expires in **5 minutes** (300 seconds)
3. Sends OTP via **Infobip WhatsApp API** to the provided phone number

Success Response `200 OK` :

```
{
  "message": "OTP sent to your WhatsApp"
}
```

3.2 POST `/v1/phone/verify-otp` — Verify OTP

Auth: Required (user role)

Rate Limit: None

Request Body (JSON):

```
{
  "phone": "+212612345678",
  "otp": "482931"
}
```

Field	Type	Required	Description
<code>phone</code>	string	Yes	Same phone used in send-otp
<code>otp</code>	string	Yes	6-digit code received via WhatsApp

How it works:

1. Looks up OTP from Redis by key `otp:{phone}`
2. If not found → expired. If mismatch → invalid.
3. On success: deletes OTP from Redis, saves `phone` to the authenticated user's profile.

Success Response `200 OK` :

```
{
  "message": "Phone number verified and saved successfully"
}
```

Error Responses:

- `400` — `{ "message": "OTP expired or not found. Request a new one." }`
- `400` — `{ "message": "Invalid OTP" }`
- `401` — `{ "message": "Authentication required" }`

4. Categories (`/v1/category`)

4.1 GET `/v1/category` — List Active Categories (Public)

Auth: None

Rate Limit: None

Success Response `200 OK` :

```
{
  "message": "Categories",
  "data": [
    {
      "id": 1,
      "en_name": "Electronics",
      "ar_name": "إلكترونيات",
      "fr_name": "Électronique",
      "imageUrl": "https://res.cloudinary.com/.../categories/abc.jpg",
      "addedByEmail": "admin@example.com",
      "isActive": true,
      "createdAt": "2026-06-14T10:00:00.000Z",
      "updatedAt": "2026-06-14T10:00:00.000Z",
      "subcategories": [
        {
          "id": 1,
          "en_name": "Mobile Phones",
          "isActive": true,
          "products": [
            { "id": 1, "en_name": "iPhone 15", "isActive": true }
          ]
        }
      ]
    }
  ]
}
```

Only returns categories where `isActive: true`. Subcategories and products are also filtered to active ones.

4.2 GET `/v1/category/:id/subcategories` — Get Subcategories by Category (Public)

Auth: None

Success Response `200 OK` :

```
{
  "message": "Subcategories",
  "data": [
    {
      "id": 1,
      "en_name": "Mobile Phones",
      "categoryId": 1,
      "isActive": true,
      "products": [...]
    }
  ]
}
```

Error Responses:

- 404 — { "message": "Category not found" }

4.3 POST /v1/category/admin — Create Category (Admin)

Auth: Required (admin or superAdmin)

Content-Type: multipart/form-data

Request Fields:

Field	Type	Required	Description
en_name	string	Yes	English name (unique)
ar_name	string	No	Arabic name (unique)
fr_name	string	No	French name (unique)
image	file	No	Category image (JPEG, PNG, GIF, WebP — max 2MB)

Success Response 201 Created :

```
{
  "message": "Category created",
  "data": {
    "id": 1,
    "en_name": "Electronics",
    "ar_name": "إلكترونيات",
    "fr_name": "Electronique",
    "imageUrl": "https://res.cloudinary.com/.../categories/abc.jpg",
    "addedByEmail": "admin@example.com",
    "isActive": true,
    "updatedAt": "2026-06-14T10:00:00.000Z",
    "createdAt": "2026-06-14T10:00:00.000Z"
  }
}
```

Error Responses:

- 400 — { "message": "Category already exists" }
-

4.4 PUT /v1/category/admin/:id — Update Category (Admin)

Auth: Required (admin or superAdmin)

Content-Type: multipart/form-data

Request Fields (all optional):

Field	Type	Description
en_name	string	New English name
ar_name	string	New Arabic name (send "" to clear)
fr_name	string	New French name (send "" to clear)
image	file	New category image (replaces old one)
imageUrl	string	Set image URL directly
addedByEmail	string	Change admin email

Success Response 200 OK (same shape as create)

How it works: If a new image is uploaded, the old image is deleted from Cloudinary.

Error Responses:

- 404 — { "message": "Category not found" }
 - 400 — { "message": "No fields to update" }
-

4.5 GET /v1/category/admin — List All Categories (Admin)

Auth: Required (admin or superAdmin)

Includes inactive categories, subcategories, and products.

Success Response 200 OK :

```
{
  "message": "Categories",
  "data": [
    {
      "id": 1,
      "en_name": "Electronics",
      "isActive": true,
      "subcategories": [
        {
          "id": 1,
          "en_name": "Mobile Phones",
          "products": [ ... ]
        }
      ]
    }
  ]
}
```

4.6 GET `/v1/category/admin/:id` — Get Category by ID (Admin)

Auth: Required (admin or superAdmin)

4.7 DELETE `/v1/category/admin/:id` — Soft Delete Category (Admin)

Auth: Required (admin or superAdmin)

Cascade behavior: Deleting a category also sets `isActive: false` on:

- All subcategories belonging to this category
- All products belonging to those subcategories

Success Response `200 OK` :

```
{
  "message": "Category soft deleted"
}
```

Error Responses:

- `404` — `{ "message": "Category not found" }`
-

5. Subcategories (`/v1/subcategory`)

5.1 GET /v1/subcategory — List Active Subcategories (Public)

Auth: None

Success Response 200 OK :

```
{
  "message": "Subcategories",
  "data": [
    {
      "id": 1,
      "en_name": "Mobile Phones",
      "ar_name": "هواتف محمولة",
      "fr_name": "Téléphones portables",
      "imageUrl": "https://res.cloudinary.com/.../subcategories/abc.jpg",
      "categoryId": 1,
      "addedByEmail": "admin@example.com",
      "isActive": true,
      "createdAt": "...",
      "updatedAt": "...",
      "products": [
        {
          "id": 1,
          "en_name": "iPhone 15",
          "isActive": true,
          ...
        }
      ]
    }
  ]
}
```

5.2 GET /v1/subcategory/:id — Get Active Subcategory by ID (Public)

Auth: None

5.3 GET /v1/subcategory/admin — List All Subcategories (Admin)

Auth: Required (admin or superAdmin)

Includes inactive subcategories.

5.4 GET /v1/subcategory/admin/:id — Get Subcategory by ID (Admin)

Auth: Required (admin or superAdmin)

5.5 POST /v1/subcategory/admin — Create Subcategory (Admin)

Auth: Required (admin or superAdmin)

Content-Type: multipart/form-data

Request Fields:

Field	Type	Required	Description
en_name	string	Yes	Unique English name
ar_name	string	No	Arabic name
fr_name	string	No	French name
categoryId	number	Yes	Parent category ID
image	file	No	Subcategory image (max 2MB)

Success Response 201 Created (same shape as category create)

Error Responses:

- 404 — { "message": "Category not found" }
 - 400 — { "message": "Subcategory already exists" }
-

5.6 PUT /v1/subcategory/admin/:id — Update Subcategory (Admin)

Auth: Required (admin or superAdmin)

Content-Type: multipart/form-data

Optional fields: en_name, ar_name, fr_name, categoryId, imageURL, image, addedByEmail

5.7 DELETE /v1/subcategory/admin/:id — Soft Delete Subcategory (Admin)

Auth: Required (admin or superAdmin)

Cascade behavior: Sets `isActive: false` on all products belonging to this subcategory.

Success Response `200 OK` :

```
{
  "message": "Subcategory soft deleted"
}
```

6. Products (`/v1/products`)

6.1 GET `/v1/products` — List Active Products (Public)

Auth: None

Rate Limit: None

Query Parameters:

Param	Type	Default	Description
<code>page</code>	integer	1	Page number (min 1)
<code>limit</code>	integer	10	Items per page (1–100)
<code>minPrice</code>	number	—	Minimum price filter
<code>maxPrice</code>	number	—	Maximum price filter
<code>sort</code>	string	—	Sort format: <code>price_asc</code> , <code>price_desc</code> , <code>name_asc</code> , <code>name_desc</code>

Example request:

```
GET /v1/products?page=1&limit=20&minPrice=100&maxPrice=1000&sort=price_asc
```

Success Response `200 OK` :

```
{
  "message": "Products",
  "data": {
    "products": [
      {
        "id": 1,
        "en_name": "iPhone 15",
        "ar_name": "15 آيفون",
        "fr_name": "iPhone 15",
        "en_description": "Latest Apple smartphone",
        "ar_description": "أحدث هاتف من أبل",
        "fr_description": "Dernier smartphone Apple",
        "price": 999.99,
        "discount": 10,
        "stock": 50,
        "imageURLs": "[\"https://res.cloudinary.com/.../products/img1.jpg\", \"https://res.cloudinary.com/.../products/img2.jpg\"]",
        "subcategoryId": 1,
        "addedByEmail": "admin@example.com",
        "isActive": true,
        "currency": "EGP"
      }
    ],
    "total": 45,
    "page": 1,
    "limit": 20,
    "currency": "EGP"
  }
}
```

Note: `imageURLs` is a JSON string — parse it on the client to get the array of URLs.

`discount` is a percentage (0–100). The effective price = $\text{price} - (\text{price} * \text{discount} / 100)$.

6.2 GET `/v1/products/:id` — Get Active Product by ID (Public)

Auth: None

Success Response `200 OK` :

```
{
  "message": "Product",
  "data": {
    "id": 1,
    "en_name": "iPhone 15",
    "price": 999.99,
    "discount": 10,
    "stock": 50,
    "imageURLs": "[\"https://...\"]",
    "currency": "EGP",
    ...
  }
}
```

6.3 GET `/v1/products/category/:categoryId` — Products by Category (Public)

Auth: None

Supports same query parameters as `GET /products` (page, limit, minPrice, maxPrice, sort).

How it works: Finds all active subcategories belonging to the given category, then returns products in those subcategories.

Success Response `200 OK` :

```
{
  "message": "Products",
  "data": {
    "products": [],
    "total": 0,
    "currency": "EGP"
  }
}
```

Returns `products: []` if no subcategories found for the category.

6.4 GET `/v1/products/subcategory/:subcategoryId` — Products by Subcategory (Public)

Auth: None

Supports same query parameters.

6.5 POST /v1/products/admin — Create Product (Admin)

Auth: Required (admin or superAdmin)

Content-Type: multipart/form-data

Request Fields:

Field	Type	Required	Description
en_name	string	Yes	Unique English name
en_description	string	Yes	English description
price	number	Yes	Price (min 1)
subcategoryId	number	Yes	Parent subcategory ID
ar_name	string	No	Arabic name
fr_name	string	No	French name
ar_description	string	No	Arabic description
fr_description	string	No	French description
discount	number	No	Discount percentage (0–100, default 0)
stock	number	No	Stock quantity (min 0, default 0)
images[]	file[]	No	Up to 5 images (JPEG, PNG, GIF, WebP — max 2MB each)

Success Response 201 Created :

```
{
  "message": "Product created",
  "data": {
    "id": 1,
    "en_name": "iPhone 15",
    "imageURLs": "[\"https://cloudinary.com/.../img1.jpg\", \"https://cloudinary.com/.../img2.jpg\"]",
    "currency": "EGP",
    ...
  }
}
```

6.6 PUT /v1/products/admin/:id — Update Product (Admin)

Auth: Required (admin or superAdmin)

Content-Type: `multipart/form-data`

How it works with images:

- If new `images[]` are uploaded, **all existing images are deleted from Cloudinary** and replaced.
 - If `imageURLs` string is sent (without files), it sets the image URLs directly.
 - If neither is sent, existing images are kept.
-

6.7 GET `/v1/products/admin` — List All Products (Admin)

Auth: Required (admin or superAdmin)

Includes inactive products.

6.8 GET `/v1/products/admin/:id` — Get Product by ID (Admin)

Auth: Required (admin or superAdmin)

Includes inactive products.

6.9 DELETE `/v1/products/admin/:id` — Soft Delete Product (Admin)

Auth: Required (admin or superAdmin)

Success Response `200 OK` :

```
{
  "message": "Product soft deleted"
}
```

7. Cart (`/v1/cart`)

All cart endpoints require authentication with role `user` .

7.1 POST `/v1/cart` — Add Item to Cart

Auth: Required (user)

Rate Limit: None

Request Body (JSON):

```
{
  "productId": 1,
  "quantity": 2
}
```

Behavior:

- If the product is already in the user's cart, the quantity is **increased** by the given amount (not replaced).
- Quantity cannot exceed product's `stock`.
- Validates that the product exists and is active.

Success Responses:

- `201 Created` (new item):

```
{
  "message": "Item added to cart",
  "data": {
    "id": 1,
    "userId": 1,
    "productId": 1,
    "quantity": 2,
    "updatedAt": "...",
    "createdAt": "..."
  }
}
```

- `200 OK` (existing item, quantity increased):

```
{
  "message": "Cart updated",
  "data": { ... }
}
```

Error Responses:

- `404` — { "message": "Product not found or inactive" }
 - `400` — { "message": "Requested quantity exceeds stock (50 available)" }
 - `400` — { "message": "Total quantity would exceed stock (50 available)" }
-

7.2 GET /v1/cart — View Cart

Auth: Required (user)

Success Response 200 OK :

```
{
  "message": "Cart",
  "data": {
    "items": [
      {
        "id": 1,
        "productId": 1,
        "quantity": 2,
        "product": {
          "id": 1,
          "en_name": "iPhone 15",
          "ar_name": "15 آيفون",
          "fr_name": "iPhone 15",
          "price": 999.99,
          "discount": 10,
          "stock": 50,
          "imageURLs": "[\"https://...\"]"
        },
        "subtotal": 1799.98
      }
    ],
    "total": 1799.98,
    "currency": "EGP"
  }
}
```

`subtotal` = $(\text{price} - \text{price} * \text{discount} / 100) \times \text{quantity}$

`total` = sum of all subtotals

7.3 PUT /v1/cart/:productId — Update Cart Item Quantity

Auth: Required (user)

Request Body (JSON):

```
{
  "quantity": 5
}
```

Success Response 200 OK :

```
{
  "message": "Cart updated",
  "data": { ... }
}
```

Error Responses:

- 404 — { "message": "Item not found in cart" }
- 400 — { "message": "Product is no longer available" }
- 400 — { "message": "Requested quantity exceeds stock (50 available)" }

7.4 DELETE /v1/cart/:productId — Remove Item from Cart

Auth: Required (user)

Success Response 200 OK :

```
{
  "message": "Item removed from cart"
}
```

7.5 DELETE /v1/cart — Clear Entire Cart

Auth: Required (user)

Success Response 200 OK :

```
{
  "message": "Cart cleared"
}
```

8. Wishlist (/v1/wishlist)

All wishlist endpoints require authentication with role `user` .

8.1 POST /v1/wishlist/:productId — Add to Wishlist

Auth: Required (user)

Path Parameters:

Param	Type	Description
productId	number	Product ID to add

Success Response 201 Created :

```
{
  "message": "Added to wishlist",
  "data": {
    "id": 1,
    "userId": 1,
    "productId": 1,
    "updatedAt": "...",
    "createdAt": "..."
  }
}
```

Error Responses:

- 404 — { "message": "Product not found or inactive" }
- 400 — { "message": "Product already in wishlist" }

8.2 GET /v1/wishlist — View Wishlist

Auth: Required (user)

Success Response 200 OK :

```
{
  "message": "wishlist",
  "data": [
    {
      "id": 1,
      "userId": 1,
      "productId": 1,
      "product": {
        "id": 1,
        "en_name": "iPhone 15",
        "price": 999.99,
        "discount": 10,
        "stock": 50,
        "imageURLs": "[\"https://...\\\"]"
      }
    }
  ]
}
```

8.3 DELETE `/v1/wishlist/:productId` — Remove from Wishlist

Auth: Required (user)

Success Response `200 OK` :

```
{
  "message": "Removed from wishlist"
}
```

Error Responses:

- `404` — `{ "message": "Item not found in wishlist" }`

9. Orders & Payment (`/v1/order`)

9.1 POST `/v1/order` — Create Order

Auth: Required (user)

Rate Limit: 1 request per 60 seconds

Request Body (JSON):

```
{
  "items": [
    { "productId": 1, "quantity": 2 },
    { "productId": 3, "quantity": 1 }
  ],
  "paymentMethod": "cod",
  "shippingAddress": {
    "fullName": "John Doe",
    "phone": "+201234567890",
    "street": "123 Main St",
    "city": "Cairo",
    "state": "Cairo",
    "zipCode": "12345",
    "country": "Egypt"
  }
}
```

Field	Type	Required	Description
<code>items[].productId</code>	number	Yes	Product ID
<code>items[].quantity</code>	number	Yes	Quantity (min 1)
<code>paymentMethod</code>	string	Yes	<code>cod</code> (Cash on Delivery) or <code>card</code> (Stripe)
<code>shippingAddress</code>	object	Yes	Shipping address object

How it works (behind the scenes):

1. Stock validation & deduction: Uses a database transaction with pessimistic locking

(`t.LOCK.UPDATE`). For each item:

- Checks product exists
- Checks stock $\geq$ requested quantity
- Calculates discounted price: $\text{price} - (\text{price} * \text{discount} / 100)$
- Deducts quantity from stock

2. Total calculation:

- Subtotal = sum of (discountedPrice $\times$ quantity) for all items
- Shipping cost: **Only applied for COD orders**
 - If `SHIPPING_COST_TYPE = fixed` : shipping = `SHIPPING_COST_VALUE` (e.g. 50 EGP)
 - If `SHIPPING_COST_TYPE = percentage` : shipping = subtotal $\times$ `SHIPPING_COST_VALUE` / 100
- Grand total = subtotal + shipping cost

3. Order creation: Stores items (JSON), shipping address (JSON), amounts, payment/order status.

A) COD Payment Response `201 Created` :

```
{
  "message": "Order created",
  "data": {
    "id": 1,
    "userId": 1,
    "items": [
      {
        "productId": 1,
        "quantity": 2,
        "price": 899.99,
        "en_name": "iPhone 15",
        "ar_name": "15 آيفون",
        "fr_name": "iPhone 15"
      }
    ],
    "totalAmount": 1849.98,
    "shippingCost": 50,
    "paymentMethod": "cod",
    "paymentStatus": "pending",
    "orderStatus": "pending",
    "shippingAddress": {
      "fullName": "John Doe",
      "phone": "+201234567890",
      "street": "123 Main St",
      "city": "Cairo",
      "state": "Cairo",
      "zipCode": "12345",
      "country": "Egypt"
    },
    "currency": "EGP",
    "createdAt": "...",
    "updatedAt": "..."
  }
}
```

B) Card (Stripe) Payment Response 201 Created :

```
{
  "message": "Redirect to payment",
  "data": {
    "url": "https://checkout.stripe.com/c/pay/cs_test_...",
    "orderId": 1
  }
}
```

Frontend flow: Redirect the user to the `url` . After successful payment, Stripe redirects to:

- **Success:** `{BASE_URL}/complete?session_id={CHECKOUT_SESSION_ID}`
- **Cancel:** `{BASE_URL}/cancel`

Error Responses:

- **400** — `{ "message": "Insufficient stock for iPhone 15. Available: 5, requested: 10" }`
- **404** — `{ "message": "Product with id 999 not found" }`

9.2 GET /v1/order/mine — Get My Orders

Auth: Required (user)

Success Response 200 OK :

```
{
  "message": "Orders",
  "data": {
    "orders": [
      {
        "id": 1,
        "totalAmount": 1849.98,
        "orderStatus": "pending",
        "paymentStatus": "pending",
        "paymentMethod": "cod",
        "items": [ ... ],
        "shippingAddress": { ... },
        "createdAt": "..."
      }
    ],
    "currency": "EGP"
  }
}
```

Orders are returned newest first.

9.3 GET /v1/order/:id — Get Order by ID

Auth: Required (user, admin, or superAdmin)

Access rules:

- Regular `user` can only view their own orders
- `admin` and `superAdmin` can view any order

Error Responses:

- 403 — { "message": "Access denied" }

9.4 GET /v1/order/admin — Get All Orders (Admin)

Auth: Required (admin or superAdmin)

9.5 PATCH /v1/order/admin/:id — Update Order Status (Admin)

Auth: Required (admin or superAdmin)

Request Body (JSON):

```
{
  "orderStatus": "shipped",
  "paymentStatus": "paid"
}
```

Field	Type	Required	Allowed Values
orderStatus	string	Yes	pending , processing , shipped , delivered , cancelled
paymentStatus	string	No	pending , paid , failed

Success Response 200 OK :

```
{
  "message": "Order updated",
  "data": {
    "id": 1,
    "orderStatus": "shipped",
    "paymentStatus": "paid",
    "items": [ ... ],
    "shippingAddress": { ... },
    "currency": "EGP"
  }
}
```

9.6 POST /v1/order/webhook — Stripe Webhook

Auth: None (Stripe sends this)

Content-Type: application/json (raw body)

Purpose: Receives Stripe events. On checkout.session.completed , updates the order:

- paymentStatus → paid
- orderStatus → processing

Success Response 200 OK :

```
{
  "received": true
}
```

How to test locally:

- Use [Stripe CLI](https://stripe.com/docs/stripe-cli) (<https://stripe.com/docs/stripe-cli>) to forward webhooks:

```
stripe listen --forward-to localhost:3007/v1/order/webhook
```

- Set the generated webhook signing secret as `STRIPE_WEBHOOK_SECRET` in your `.env`

10. Security Logs (`/v1/admin`)

10.1 GET `/v1/admin/security/logs`

Auth: Required (superAdmin only)

Success Response `200 OK` :

```
{
  "message": "Security logs",
  "data": [
    {
      "timestamp": "2026-06-14T10:15:30",
      "timeZone": "Africa/Cairo",
      "ipAddress": "192.168.1.45",
      "type": "SECURITY_ALERT",
      "potentialInjection": "SELECT * FROM users",
      "message": "Possible injection attempt detected",
      "level": "warn"
    }
  ]
}
```

Returns parsed entries from `security.log` . Returns empty array if file doesn't exist or is empty.

Error Responses:

- `403` — `{ "message": "Access denied" }` (if not superAdmin)

11. Error Handling & Auth System

Global Response Format

All API responses follow a consistent format:

Success:

```
{
  "message": "Action completed",
  "data": { ... }
}
```

Error:

```
{
  "message": "Error description",
  "error": [ ... ] // optional, only for validation errors
}
```

HTTP Status Codes Used

Code	Meaning
200	Success (GET, PUT, PATCH, DELETE)
201	Created (POST)
400	Validation error / Bad request
401	Unauthorized / Invalid token
403	Forbidden (wrong role or blocked IP)
404	Resource not found
500	Internal server error

Authentication Flow

1. **Signup** → User receives verification email → User clicks link → `isActive: true`
2. **Login** → Server returns `accessToken` (30min) + `refreshToken` (1 year)
3. **Authenticated requests** → Include header: `Authorization: <role> <token>`
4. **Token verification** → Server extracts bearer role, selects role-specific secret, verifies JWT

Role-Based Access Control

Role	Access Level
user	Browse products, manage cart/wishlist, place orders, view own orders
admin	CRUD categories, subcategories, products; view all orders; update order status
superAdmin	All admin privileges + view security logs

Security: Injection Detection

All input validation uses a custom `detectInjection` Joi rule that scans for:

- **SQL keywords:** `SELECT`, `DROP`, `UNION`, `INSERT`, `UPDATE`, `DELETE`
- **NoSQL operators:** `$ne`, `$eq`, `$gt`, `$lt`, `$regex`
- **Comment syntax:** `--`, `;`

When detected:

1. The request is rejected with `400 Bad Request`
2. The incident is **logged** to `security.log` via Winston
3. The attacker's **IP is permanently blocked** in Redis (key: `blocked:<ip>`)
4. All subsequent requests from that IP receive `403 Access denied. IP is permanently blocked.`
5. To unblock: `DEL blocked:<ip>` in Redis

Stripe Payment Flow (Card Orders)

```
User → POST /v1/order (paymentMethod: "card")
  → Backend: validates stock, deducts stock, creates order (status: pending)
  → Backend: creates Stripe Checkout Session with line items + metadata (orderId)
  → Response: { url: "https://checkout.stripe.com/...", orderId: 1 }

Frontend → redirects user to Stripe Checkout URL
User → enters card details on Stripe hosted page
User → completes/cancels payment

Stripe → POST /v1/order/webhook (checkout.session.completed)
  → Backend: verifies webhook signature
  → Backend: updates order → paymentStatus: "paid", orderStatus: "processing"
  → Response: { received: true }

Stripe → redirects user to {BASE_URL}/complete?session_id={CHECKOUT_SESSION_ID}
```

Image Upload

Two modes configurable via `Image_TYPE` :

Mode	Storage	Description
<code>CLOUDINARY</code> (default)	Cloudinary	Images uploaded to Cloudinary CDN. Deletes old images on update.
<code>SERVER</code>	Local server	Images saved to <code>uploads/</code> folder. Served via <code>{SERVER}/uploads/{folder}/{filename}</code>

Supported formats: JPEG, PNG, GIF, WebP (validated by `file-type` package, not just extension).

Max upload size: **2MB** per file (configurable via `upload(2)` parameter).

12. Database Models

User (`users` table)

Field	Type	Description
<code>id</code>	INTEGER (PK, auto)	Unique ID
<code>name</code>	STRING (unique)	Username
<code>email</code>	STRING (unique)	Email address
<code>password</code>	STRING (nullable)	Hashed password (null for Google OAuth users)
<code>isActive</code>	BOOLEAN	Email verified / soft delete
<code>imageUrl</code>	STRING (nullable)	Profile image URL
<code>role</code>	ENUM('admin','user','superAdmin')	User role
<code>phone</code>	STRING (nullable)	Phone number
<code>googleId</code>	STRING (nullable)	Google OAuth ID
<code>isBlocked</code>	BOOLEAN	Manually blocked flag

Category (categories table)

Field	Type	Description
id	INTEGER (PK, auto)	Unique ID
en_name	STRING (unique)	English name
ar_name	STRING (unique, nullable)	Arabic name
fr_name	STRING (unique, nullable)	French name
imageUrl	STRING (nullable)	Category image URL
addedByEmail	STRING	Admin email who created it
isActive	BOOLEAN	Soft delete flag

Relations: hasMany → Subcategory

Subcategory (subcategories table)

Field	Type	Description
id	INTEGER (PK, auto)	Unique ID
en_name	STRING (unique)	English name
ar_name	STRING (unique, nullable)	Arabic name
fr_name	STRING (unique, nullable)	French name
imageUrl	STRING (nullable)	Subcategory image URL
categoryId	INTEGER (FK)	Parent category ID
addedByEmail	STRING	Admin email who created it
isActive	BOOLEAN	Soft delete flag

Relations: belongsTo → Category , hasMany → Product

Product (products table)

Field	Type	Description
id	INTEGER (PK, auto)	Unique ID
en_name	STRING (unique)	English name
ar_name	STRING (unique, nullable)	Arabic name
fr_name	STRING (unique, nullable)	French name
en_description	TEXT	English description
ar_description	TEXT (nullable)	Arabic description
fr_description	TEXT (nullable)	French description
price	DECIMAL	Base price
discount	INTEGER	Discount percentage (0–100)
stock	INTEGER	Available stock quantity
imageURLs	TEXT	JSON array of image URLs
subcategoryId	INTEGER (FK)	Parent subcategory ID
addedByEmail	STRING	Admin email who created it
isActive	BOOLEAN	Soft delete flag

Relations: belongsTo → Subcategory , hasMany → Cart , hasMany → Wishlist

Cart (carts table)

Field	Type	Description
id	INTEGER (PK, auto)	Unique ID
userId	INTEGER	User ID (owner)
productId	INTEGER (FK)	Product ID
quantity	INTEGER	Quantity (min 1)

Relations: belongsTo → Product

Wishlist (wishlists table)

Field	Type	Description
<code>id</code>	INTEGER (PK, auto)	Unique ID
<code>userId</code>	INTEGER	User ID (owner)
<code>productId</code>	INTEGER (FK)	Product ID

Relations: `belongsTo` → Product

Order (`orders` table)

Field	Type	Description
<code>id</code>	INTEGER (PK, auto)	Unique ID
<code>userId</code>	INTEGER	User ID (owner)
<code>items</code>	TEXT	JSON: array of { productId, quantity, price, en_name, ar_name, fr_name }
<code>totalAmount</code>	DECIMAL(10,2)	Grand total (subtotal + shipping)
<code>shippingCost</code>	DECIMAL(10,2)	Shipping cost (0 for card orders)
<code>paymentMethod</code>	ENUM('cod', 'card')	Payment method
<code>paymentStatus</code>	ENUM('pending', 'paid', 'failed')	Payment status
<code>orderStatus</code>	ENUM('pending', 'processing', 'shipped', 'delivered', 'cancelled')	Order fulfillment status
<code>shippingAddress</code>	TEXT	JSON: { fullName, phone, street, city, state, zipCode, country }

Quick Reference: All Endpoints

#	Method	Endpoint	Auth	Description
1	POST	<code>/v1/auth/signup</code>	—	Register new user
2	POST		—	Login with

		/v1/auth/login		email/password
3	GET	/v1/auth/google/signup	—	Google OAuth signup
4	GET	/v1/auth/google/signup/callback	—	Google signup callback
5	GET	/v1/auth/google/login	—	Google OAuth login
6	GET	/v1/auth/google/login/callback	—	Google login callback
7	GET	/v1/auth/verify-email/:token	—	Verify email
8	POST	/v1/auth/resend-verify-email	R60	Resend verification email
9	POST	/v1/auth/forgot-password	R60	Send password reset email
10	POST	/v1/auth/reset-password/:token	R60	Reset password
11	GET	/v1/user/profile	User	Get profile
12	PUT	/v1/user/profile	User	Update profile
13	POST	/v1/user/upload-avatar	User	Upload avatar
14	POST	/v1/phone/send-otp	User	Send OTP via WhatsApp
15	POST	/v1/phone/verify-otp	User	Verify OTP
16	GET	/v1/category	—	List active categories
17	GET	/v1/category/:id/subcategories	—	Get subcategories by category
18	POST	/v1/category/admin	Admin	Create category
19	PUT	/v1/category/admin/:id	Admin	Update category
20	GET	/v1/category/admin	Admin	List all categories
21	GET	/v1/category/admin/:id	Admin	Get category by ID
22	DELETE	/v1/category/admin/:id	Admin	Soft delete category
23	GET	/v1/subcategory	—	List active

				subcategories
24	GET	/v1/subcategory/:id	—	Get active subcategory
25	POST	/v1/subcategory/admin	Admin	Create subcategory
26	PUT	/v1/subcategory/admin/:id	Admin	Update subcategory
27	GET	/v1/subcategory/admin	Admin	List all subcategories
28	GET	/v1/subcategory/admin/:id	Admin	Get subcategory by ID
29	DELETE	/v1/subcategory/admin/:id	Admin	Soft delete subcategory
30	GET	/v1/products	—	List active products
31	GET	/v1/products/:id	—	Get active product
32	GET	/v1/products/category/:categoryId	—	Products by category
33	GET	/v1/products/subcategory/:subcategoryId	—	Products by subcategory
34	POST	/v1/products/admin	Admin	Create product
35	PUT	/v1/products/admin/:id	Admin	Update product
36	GET	/v1/products/admin	Admin	List all products
37	GET	/v1/products/admin/:id	Admin	Get product by ID
38	DELETE	/v1/products/admin/:id	Admin	Soft delete product
39	POST	/v1/cart	User	Add item to cart
40	GET	/v1/cart	User	View cart
41	PUT	/v1/cart/:productId	User	Update cart quantity
42	DELETE	/v1/cart/:productId	User	Remove item from cart
43	DELETE	/v1/cart	User	Clear cart
44	POST	/v1/wishlist/:productId	User	Add to wishlist
45	GET	/v1/wishlist	User	View wishlist

46	DELETE	/v1/wishlist/:productId	User	Remove from wishlist
47	POST	/v1/order	User (R60)	Create order (COD/Card)
48	GET	/v1/order/mine	User	Get my orders
49	GET	/v1/order/:id	User	Get order by ID
50	GET	/v1/order/admin	Admin	Get all orders
51	PATCH	/v1/order/admin/:id	Admin	Update order status
52	POST	/v1/order/webhook	—	Stripe webhook
53	GET	/v1/admin/security/logs	SuperAdmin	View security logs

R60 = rate-limited: 1 request per 60 seconds

Tech Stack Summary

Layer	Technology
Runtime	Node.js + TypeScript
Framework	Express 5
ORM	Sequelize 6
Database	MSSQL (SQL Server)
Cache / OTP	Redis (Upstash)
Auth	JWT (role-based secrets) + Passport (Google OAuth)
Validation	Joi (with custom injection detection)
Image Upload	Cloudinary (primary) or local server
Payments	Stripe Checkout
WhatsApp OTP	Infobip API
Logging	Winston (<code>security.log</code>)
Email	Nodemailer (Gmail SMTP)
File Type Validation	<code>file-type</code> package (reads actual bytes)
Testing	Jest (unit) + Cypress (E2E, ~106 tests)

Running the Project

```
# 1. Install dependencies
npm install

# 2. Configure environment
# Edit config/.env with your settings (database, Redis, Stripe, etc.)

# 3. Start in development mode
npm run dev

# 4. Server runs on http://localhost:3007
```

Available Scripts

Command	Description
<code>npm run dev</code>	Start dev server with hot reload
<code>npm run build</code>	Compile TypeScript
<code>npm run start</code>	Run compiled JS
<code>npm run test</code>	Run Jest tests
<code>npm run cy:run</code>	Run Cypress E2E tests headlessly
<code>npm run lint</code>	ESLint
<code>npm run format</code>	Prettier